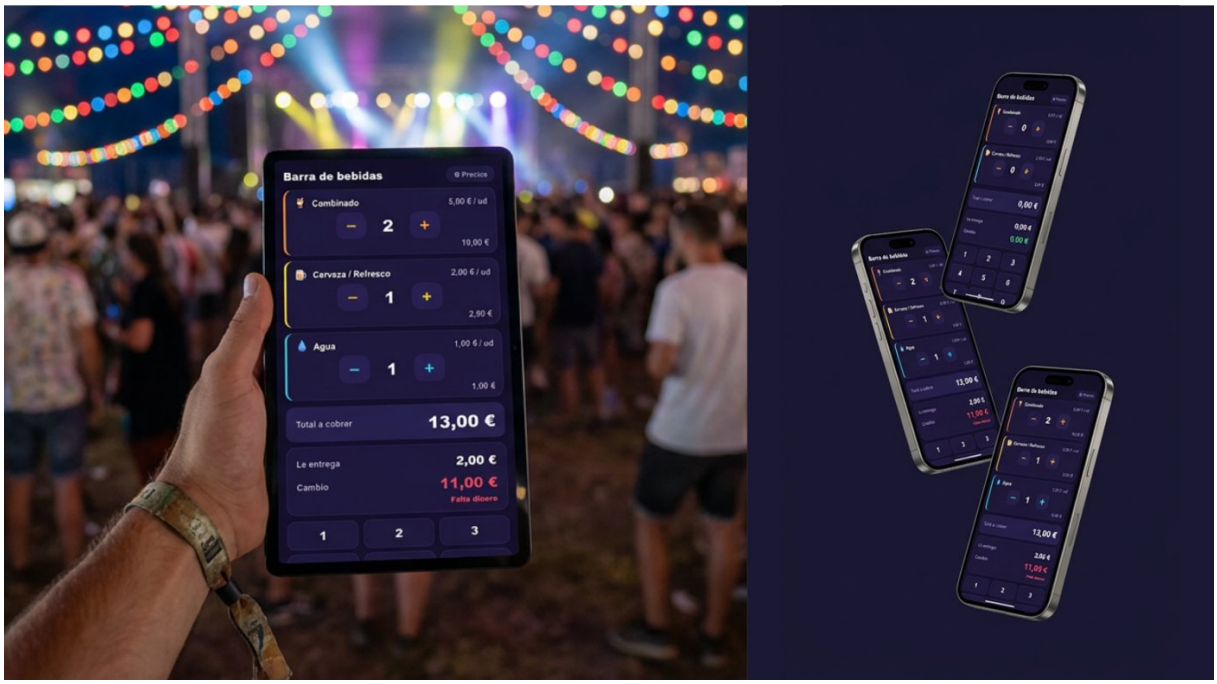


TPV Barra

Punto de venta en el móvil para la barra de un festival

Eva Gallardo Romero

28 de julio de 2026

Índice

Enlace al repositorio de GitHub **2**

1 Descripción general **2**

2 Diagrama de clases **2**

3 Diseño **3**
3.1 Venta 3
3.2 Precios 3

4 Requisitos del sistema **4**
4.1 Requisitos funcionales 4
4.2 Requisitos no funcionales 5

5 Documentación técnica **5**
5.1 script.js: catálogo y estado 5
5.2 script.js: dinero 6
5.3 script.js: precios guardados 6
5.4 script.js: interfaz 7
5.5 sw.js 8

Enlace al repositorio de GitHub

Puedes consultar el proyecto completo en el siguiente enlace: tpv-barra-festival.

La aplicación está publicada en evagr11.github.io/tpv-barra-festival y la documentación del código generada con JSDoc, en evagr11.github.io/tpv-barra-festival/docs.

1 Descripción general

TPV Barra es una aplicación web que hace de **punto de venta táctil** en la barra de un evento. Lleva la cuenta de lo que pide cada cliente, suma el total y calcula el **cambio** a partir del dinero que entrega. La usé de verdad para cobrar en la barra de **Contempopranea** (Don Benito).

Ahora tiene la carta de ese festival: **11 productos en tres bloques** (vaso, agua y Monster; y cerveza, tinto de verano, refresco y combinado en vasos de 400 y 900 ml). Toda la carta sale de un único array en `script.js`, así que adaptarla a otro evento es cambiar unas líneas.

Está hecha en **HTML, CSS y JavaScript** sin frameworks y es una **PWA**: se instala en la pantalla de inicio con su icono y un *service worker* la guarda en el móvil, así que funciona **sin cobertura**, algo imprescindible en un festival. Los precios se cambian desde la propia app y se guardan en el dispositivo.

Lo más delicado de un TPV es el cambio: un error de redondeo puede hacer que la caja no cuadre. Por eso **todo el dinero se maneja en céntimos**, como números enteros, y solo se convierte a euros para mostrarlo.

2 Diagrama de clases

No hay clases: todo el código de la interfaz está en una función que se ejecuta al cargar (IIFE), con el estado en tres variables y una función por cada cosa que se pinta. El *service worker* va en un archivo aparte.

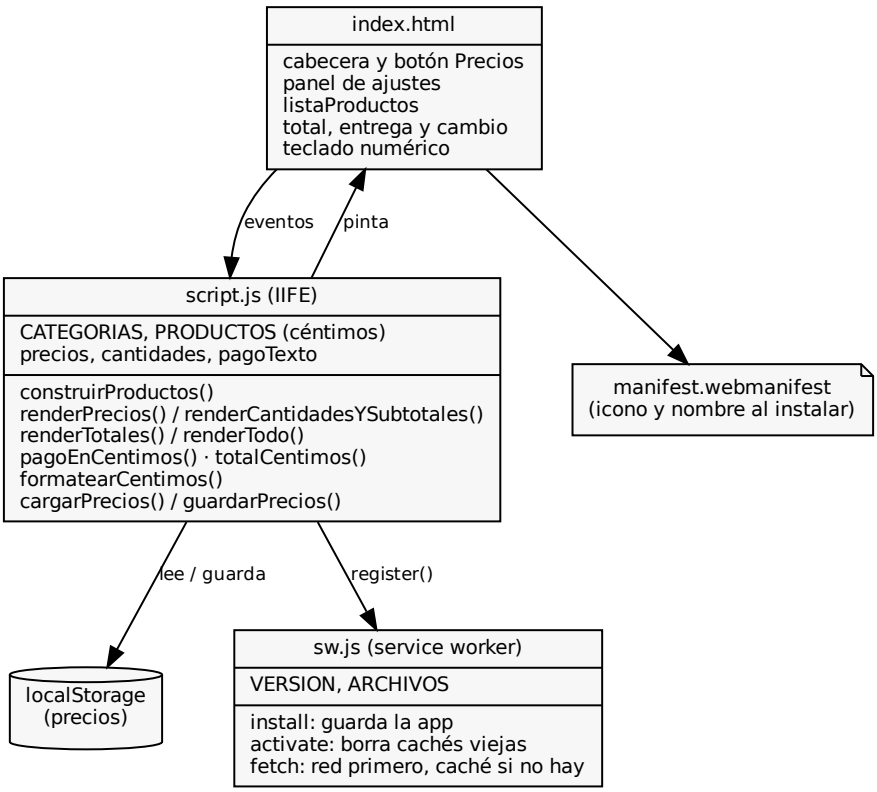


Figura 1: Estructura de TPV Barra

3 Diseño

Pensada para usarla con una mano y con prisa: fondo oscuro para no deslumbrar de noche, botones grandes y cada bloque de la carta con su color (azul para *La bebida*, amarillo para el vaso de 400 ml y naranja para el de 900 ml). El zoom está bloqueado para que un toque rápido no amplíe la pantalla.

3.1 Venta

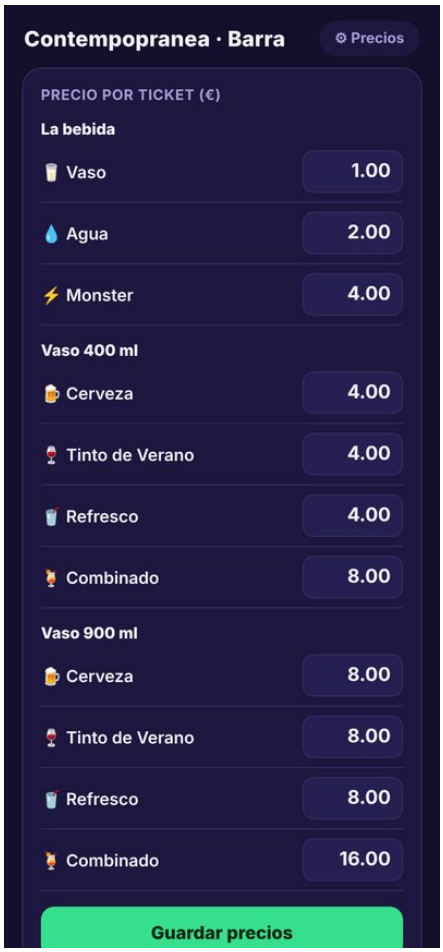
Arriba, el botón *Siguiente cliente · Limpiar todo*. Cada producto tiene su icono, su precio, los botones − y +, la cantidad y el subtotal. Debajo, el **total a cobrar**, lo que **entrega** el cliente y el **cambio**, que se pone en rojo con el aviso *Falta dinero* si no llega. El importe se escribe con un **teclado tipo datáfono**.

3.2 Precios

El botón *Precios* abre un panel con el precio de cada producto por bloques y el botón *Guardar precios*.



3.1. Venta en curso



3.2. Ajuste de precios

Figura 2: Capturas de TPV Barra en el móvil

4 Requisitos del sistema

4.1 Requisitos funcionales

ID	Requisito funcional
RF01	La app debe mostrar los productos de la carta agrupados por **bloques**, con botones para sumar y restar.
RF02	La cantidad de un producto no puede bajar de 0.
RF03	Debe calcular el **subtotal** de cada producto y el **total** de la ronda.
RF04	El dinero entregado se debe escribir con un teclado numérico, con como mucho dos decimales.
RF05	Debe calcular el **cambio** y avisar con *Falta dinero* si el pago no llega al total.

ID	Requisito funcional
RF06	Un botón debe dejar la venta a cero para el **siguiente cliente**.
RF07	Los precios se deben poder **cambiar** desde la app y seguir guardados al cerrarla.
RF08	La app debe poder **instalarse** y abrir **sin conexión**.

4.2 Requisitos no funcionales

ID	Requisito no funcional
RNF01	Debe estar hecha en **HTML, CSS y JavaScript** sin frameworks ni dependencias.
RNF02	Todo el dinero se debe calcular en **céntimos enteros** para que no haya errores de redondeo.
RNF03	Los importes se muestran con el formato español (`es-ES`): coma decimal y símbolo del euro.
RNF04	Diseño **mobile first**, probado en Safari para iPhone, sin zoom accidental.
RNF05	Debe publicarse en **GitHub Pages** y funcionar como PWA (manifest y service worker).
RNF06	El código debe tener comentarios **JSDoc**.

5 Documentación técnica

Este anexo agrupa la documentación por archivos, separándola en la **ubicación** donde se encuentra, una **descripción general** y las **funciones** explicadas de manera individual con su **código** adjunto y comentado. Se omiten los comentarios JSDoc, que repiten la descripción.

5.1 script.js: catálogo y estado

5.1.1 Ubicación

`tpv-barra-festival/script.js`

Descripción general:
La carta, los precios guardados, las cantidades de la venta actual y lo que ha escrito el cliente como pago.

5.1.2 Funciones

CATEGORIAS y PRODUCTOS **Descripción:** Los bloques y los productos, con el precio en céntimos. Se muestran algunos productos; en el código cada uno lleva también su icono.

Código comentado:

```
const CATEGORIAS = [
  { id: "bebida", titulo: "La bebida" },
  { id: "v400",   titulo: "Vaso 400 ml" },
  { id: "v900",   titulo: "Vaso 900 ml" }
];

const PRODUCTOS = [
  { id: "vaso",       categoria: "bebida", nombre: "Vaso",      precio: 100 },
  { id: "agua",       categoria: "bebida", nombre: "Agua",      precio: 200 },
  { id: "cerveza400", categoria: "v400",   nombre: "Cerveza",   precio: 400 },
  { id: "combi900",   categoria: "v900",   nombre: "Combinado", precio: 1600 },
  // ... 11 productos en total
];

```

```
let precios = cargarPrecios();                                    // céntimos
let cantidades = IDS.reduce((acc, id) => (acc[id] = 0, acc), {});
let pagoTexto = "";                                               // lo tecleado, p. ej.
    "20,5"
```

5.2 script.js: dinero

5.2.1 Ubicación

`tpv-barra-festival/script.js`

Descripción general:
Conversión entre lo que se escribe, los céntimos y el texto en euros.

5.2.2 Funciones

pagoEnCentimos **Descripción:** Convierte el texto del teclado a céntimos sin pasar por decimales: `"20,5"` son 2050 céntimos.

Código comentado:

```
function pagoEnCentimos() {
  if (pagoTexto === "") return 0;
  if (!pagoTexto.includes(",")) {
    return parseInt(pagoTexto, 10) * 100;          // solo euros
  }
  const partes = pagoTexto.split(",");
  const euros = partes[0] === "" ? 0 : parseInt(partes[0], 10);
  let centimos = partes[1] || "";
  if (centimos.length === 1) centimos += "0";      // "5" son 50 céntimos
  if (centimos.length === 0) centimos = "00";
  return euros * 100 + parseInt(centimos, 10);
}
```

formatearCentimos y totalCentimos **Descripción:** Muestra un importe en euros con el formato español y suma precio por cantidad de todos los productos.

Código comentado:

```
function formatearCentimos(centimos) {
  const euros = centimos / 100;
  return euros.toLocaleString("es-ES", {
    minimumFractionDigits: 2,
    maximumFractionDigits: 2
  }) + " EUR";
}

function totalCentimos() {
  return IDS.reduce((acc, id) => acc + precios[id] * cantidades[id], 0);
}
```

5.3 script.js: precios guardados

5.3.1 Ubicación

`tpv-barra-festival/script.js`

Descripción general:
Los precios se guardan en el `localStorage` del móvil.

5.3.2 Funciones

cargarPrecios y guardarPrecios **Descripción:** Carga los precios guardados solo si están todos y son números; si no, usa los de la carta. Si el almacenamiento falla, la app sigue funcionando.

Código comentado:

```
function cargarPrecios() {
  try {
    const guardado = localStorage.getItem(CLAVE_STORAGE);
    if (guardado) {
      const obj = JSON.parse(guardado);
      if (IDS.every(id => Number.isFinite(obj[id]))) return obj;
    }
  } catch (e) { /* si falla, usamos los de por defecto */ }
  return { ...PRECIOS_POR_DEFECTO };
}

function guardarPrecios() {
  try {
    localStorage.setItem(CLAVE_STORAGE, JSON.stringify(precios));
  } catch (e) { /* almacenamiento no disponible, seguimos igualmente */ }
}
```

5.4 script.js: interfaz

5.4.1 Ubicación

`tpv-barra-festival/script.js`

Descripción general:
Las tarjetas se generan desde el array y se repintan cuando cambia algo.

5.4.2 Funciones

renderTotales **Descripción:** Pinta el total, el pago y el cambio; si el cambio es negativo, lo marca en rojo y enseña el aviso.

Código comentado:

```
function renderTotales() {
  const total = totalCentimos();
  el.totalCifra.textContent = formatearCentimos(total);

  const pagoCentimos = pagoEnCentimos();
  el.pagoCifra.textContent = formatearCentimos(pagoCentimos);

  const cambio = pagoCentimos - total;
  if (cambio < 0) {
    el.cambioFila.classList.add("negativo");
    el.cambioCifra.textContent = formatearCentimos(Math.abs(cambio));
    el.avisoInsuficiente.classList.remove("oculto");     // "Falta dinero"
  } else {
    el.cambioFila.classList.remove("negativo");
    el.cambioCifra.textContent = formatearCentimos(cambio);
    el.avisoInsuficiente.classList.add("oculto");
  }
}
```

Contadores **Descripción:** Cada botón + o − lleva el producto y la acción en atributos `data-`. La cantidad nunca baja de 0.

Código comentado:

```
document.querySelectorAll(".btn-contador").forEach(btn => {
  btn.addEventListener("click", () => {
    const id = btn.dataset.id;
    const accion = btn.dataset.accion;
    if (accion === "mas") {
      cantidades[id]++;
    } else if (accion === "menos") {
      cantidades[id] = Math.max(0, cantidades[id] - 1);
    }
    renderCantidadesYSubtotales();
```

```
    renderTotales();
  });
});
```

Teclado numérico **Descripción:** Borra, pone el importe exacto, añade la coma una sola vez o añade un número (como mucho dos decimales).

Código comentado:

```
const key = btn.dataset.key;
if (key === "borrar") {
  pagoTexto = pagoTexto.slice(0, -1);
} else if (key === "exacto") {
  pagoTexto = (totalCentimos() / 100).toFixed(2).replace(".", ",");
} else if (key === ",") {
  if (!pagoTexto.includes(",")) {
    pagoTexto = pagoTexto === "" ? "0," : pagoTexto + ",";
  }
} else {
  if (pagoTexto.includes(",")) {
    const decimales = pagoTexto.split(",")[1];
    if (decimales.length < 2) pagoTexto += key;
  } else {
    pagoTexto += key;
  }
}
renderTotales();
```

Guardar precios **Descripción:** Lee los campos del panel, admite coma o punto, redondea a céntimos y guarda.

Código comentado:

```
el.btnGuardarPrecios.addEventListener("click", () => {
  IDS.forEach(id => {
    const input = document.getElementById("precio-" + id);
    const valorEuros = parseFloat(input.value.replace(",", "."));
    if (Number.isFinite(valorEuros) && valorEuros >= 0) {
      precios[id] = Math.round(valorEuros * 100);
    }
  });
  guardarPrecios();
  el.panelAjustes.classList.add("oculto");
  renderTodo();
});
```

5.5 sw.js

5.5.1 Ubicación

`tpv-barra-festival/sw.js`

Descripción general:
El *service worker* guarda los archivos de la app para que abra sin conexión.

5.5.2 Funciones

install y fetch **Descripción:** Al instalarse guarda la lista de archivos. En cada petición intenta primero la red (para recibir cambios) y, si no hay conexión, usa la copia guardada.

Código comentado:

```
const VERSION = "barra-v1";
const ARCHIVOS = ["./", "index.html", "style.css", "script.js", "manifest.webmanifest"
    ,
  "icons/icono-192.png", "icons/icono-512.png", "icons/apple-touch-icon.png"];

self.addEventListener("install", (e) => {
```

```
6   e.waitUntil(caches.open(VERSION).then((c) => c.addAll(ARCHIVOS)).then(() => self.
        skipWaiting()));
7 });
8
9 self.addEventListener("fetch", (e) => {
10   if (e.request.method !== "GET" || new URL(e.request.url).origin !== location.origin)
         return;
11   e.respondWith(
12     fetch(e.request)
13       .then((respuesta) => {
14         if (respuesta.ok) {
15           const copia = respuesta.clone();
16           caches.open(VERSION).then((c) => c.put(e.request, copia));   // actualiza la
                 copia
17         }
18         return respuesta;
19       })
20       .catch(() => caches.match(e.request, { ignoreSearch: true }))   // sin red
21   );
22 });
```